FINDING THE REPEATED ELEMENT
IN AN ARRAY USING POINTERS AND XOR

A C program with a duplicate_element() function that finds the only repeated element of an array in a single pass

Name : Rudra Pratap Singh

SAP ID : 590032811

Batch : 16

1. Problem Statement

Write a C program that reads the size and the elements of a variable-length integer array, nums[]. The array contains exactly one element that is repeated once, and every other element appears exactly once. The array must be passed to a function named duplicate_element() using its address, and pointers must be used to pass the array between the calling function (main) and duplicate_element(). The function must find and display the repeated element in a single iteration through the array using an efficient approach such as the XOR technique.

2. Objectives

- To read an array whose size is decided by the user at run time (dynamic memory allocation).
- To pass an array to a function using pointers (call by address).
- To apply the bitwise XOR operator to solve a problem efficiently.
- To find the repeated element in one pass, with O(n) time and O(1) extra space.

3. Concepts Used

- **Pointers:** the array is passed as int *nums, and elements are accessed as *(nums + i).
- **Dynamic memory allocation:** malloc() creates an array of n integers at run time and free() releases it.
- **Bitwise XOR (^):** returns 1 only when the two bits differ.
- **Functions and loops:** a user-defined function with a single for loop.

Properties of XOR

Property	Meaning
a ^ a = 0	XOR of a number with itself is 0
a ^ 0 = a	XOR of a number with 0 is the number itself
a ^ b = b ^ a	XOR is commutative (order does not matter)
(a ^ b) ^ c = a ^ (b ^ c)	XOR is associative

4. Approach and Logic

An array of n elements contains the numbers 1 to n-1, each appearing once, and one of these numbers appears a second time. For example, for n = 5 the numbers 1, 2, 3, 4 are present and one of them is repeated.

If we XOR all the elements of the array with all the numbers from 1 to n-1, every number that is not repeated appears exactly twice (once in the array, once in the range) and cancels out to

0. The repeated number appears three times (twice in the array, once in the range), so after cancellation exactly one copy of it remains. That remaining value is the answer.

Since XOR with 0 changes nothing, the range can be taken as 0 to n-1, which is the same as the loop index i. So both XORs are done in one loop:

```
result = result ^ nums[i] ^ i;      /* for i = 0 to n-1 */
```

Algorithm

1. Start.
2. Read the size n of the array.
3. Allocate memory for n integers and read the elements into nums[].
4. Call duplicate_element(nums, n), passing the address of the array.
5. Inside the function, set result = 0.
6. For i = 0 to n-1, do result = result ^ *(nums + i) ^ i.
7. Display result as the repeated element.
8. Free the memory and stop.

Dry Run (Example 1)

Array: 1 3 4 2 2 (n = 5), initial result = 0

i	nums[i]	nums[i] ^ i	result before	result after
0	1	1	0	1
1	3	2	1	3
2	4	6	3	5
3	2	1	5	4
4	2	6	4	2

After the loop, result = 2, which is the repeated element.

5. Source Code

```
#include <stdio.h>
#include <stdlib.h>

/* Finds and displays the only repeated element of the array.
   nums : pointer to the first element of the array
   n    : number of elements in the array                    */
void duplicate_element(int *nums, int n)
{
    int result = 0;
    int i;

    /* Single pass: XOR every element with its index.
       Values 1..n-1 each appear once and the repeated value appears twice,
       so XOR-ing the elements with the numbers 0..n-1 cancels everything
       except the repeated element.                          */
    for (i = 0; i < n; i++)
    {
        result ^= *(nums + i) ^ i;
    }

    printf("Repeated element: %d\n", result);
}

int main()
{
    int n, i;
    int *nums;

    printf("Enter the size of the array: ");
    if (scanf("%d", &n) != 1 || n < 2)
    {
        printf("Invalid size. The array must have at least 2 elements.\n");
        return 1;
    }

    /* Allocate memory for a variable-length array */
    nums = (int *)malloc(n * sizeof(int));
    if (nums == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    printf("Enter %d elements (values between 1 and %d):\n", n, n - 1);
    for (i = 0; i < n; i++)
    {
        scanf("%d", nums + i);
    }

    /* Pass the array to the function using its address */
    duplicate_element(nums, n);

    free(nums);
    return 0;
}
```

6. Sample Inputs and Outputs

Test Case 1

```
Enter the size of the array: 5
Enter 5 elements (values between 1 and 4):
1 3 4 2 2
Repeated element: 2
```

Test Case 2

```
Enter the size of the array: 5
Enter 5 elements (values between 1 and 4):
3 1 3 4 2
Repeated element: 3
```

Test Case 3

```
Enter the size of the array: 6
Enter 6 elements (values between 1 and 5):
1 4 5 2 3 4
Repeated element: 4
```

7. Complexity Analysis

Measure	Complexity	Reason
Time	O(n)	The array is traversed only once
Extra space	O(1)	Only one variable (result) is used

8. Assumptions and Limitations

- The array has n elements with values from 1 to n-1, and exactly one value is repeated once.
- Every other element appears exactly once, as stated in the problem.
- The program checks that n is at least 2, but it does not verify the values entered by the user.

9. Conclusion

The program successfully finds the repeated element of an array in a single pass using the XOR technique. The array is created at run time with malloc() and passed to duplicate_element() using pointers. All three given examples were compiled and tested, and they produce the expected output. The method is efficient, using O(n) time and O(1) extra space.